1. MapReduce Fundamentals

What is MapReduce?

MapReduce is a distributed programming model used for processing **large datasets across clusters of computers**.

It is part of the **Apache Hadoop ecosystem** and enables parallel data processing.

Key Idea

Instead of processing data on a single machine, Hadoop splits data across multiple nodes and processes them **in parallel**.

Advantages

- Handles **big data (TBs / PBs)**
- **Parallel processing**
- **Fault tolerant**
- Works with **commodity hardware**

Example Use Case

Counting how many times each word appears in a large document dataset.

Example input:

Big data is powerful
Big data is scalable

Output:

Big: 2
data: 2
is: 2
powerful: 1
scalable: 1

2. Map Phase

The **Map phase** is the first stage of MapReduce.

The mapper processes input data and converts it into **key-value pairs**.

Steps

1. Input data is split into chunks.
2. Each chunk is processed by a **mapper function**.
3. Mapper outputs **(key, value)** pairs.

Example

Input:

Big data is powerful

Mapper Output:

(Big,1)
(data,1)
(is,1)
(powerful,1)

3. Shuffle and Sort Phase

After the Map phase, Hadoop automatically performs **Shuffle and Sort**.

Shuffle

Hadoop groups all values with the same key.

Example:

(Big,1)
(Big,1)
(data,1)
(data,1)

Grouped output:

Big → [1,1]
data → [1,1]

Sort

The keys are sorted before sending to reducers.

Example:

Big → [1,1]
data → [1,1]
is → [1,1]

4. Reduce Phase

The **Reducer** aggregates values for each key.

Example reducer operation:

Big → [1,1] → 2
data → [1,1] → 2

Final Output:

Big 2
data 2
is 2
powerful 1

5. Programming with Hadoop using Python

Hadoop supports **Python MapReduce using Hadoop Streaming**.

This allows developers to write **Mapper and Reducer in Python**.

Example Hands-on: Word Count Using Python MapReduce

Step 1: Create Input File

input.txt

```
big data is powerful
big data is scalable
```

Step 2: Mapper Script

mapper.py

```
import sys

for line in sys.stdin:
    words = line.strip().split()
    for word in words:
        print(f"{word}\t1")
```

Mapper Output:

```
big 1
data 1
is 1
powerful 1
```

Step 3: Reducer Script

reducer.py

```
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if word == current_word:
        current_count += count
    else:
        if current_word:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count

if current_word:
    print(f"{current_word}\t{current_count}")
```

Output:

```
big 2
data 2
is 2
powerful 1
scalable 1
```

---

6. Running Python MapReduce in Hadoop

Command:

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
-input input.txt \
-output output \
```

-mapper mapper.py \
-reducer reducer.py

---

7. Basics of YARN

YARN (Yet Another Resource Negotiator) is Hadoop's **resource management system**.

It manages:

- Cluster resources
- Job scheduling
- Application execution

Why YARN is Important

Before Hadoop 2.x:

MapReduce handled **both processing and resource management**.

With YARN:

- Resource management is **separate**
- Supports multiple processing engines

Examples:

- MapReduce
- Spark
- Flink
- Tez

YARN Architecture

Key components:

1. Resource Manager

Main cluster manager.

Responsibilities:

- Allocate cluster resources
- Schedule applications

---

2. Node Manager

Runs on each worker node.

Responsibilities:

- Monitor resource usage
- Manage containers

---

3. Application Master

Responsible for:

- Managing application execution
- Communicating with Resource Manager

YARN Workflow

1. User submits job
2. Resource Manager allocates resources
3. Application Master starts
4. Containers execute tasks
5. Results are returned

Hands-on Exercise for Students

Exercise 1

Write a MapReduce program to count:

- Number of occurrences of each **product category**.

Dataset:

Laptop Electronics
Mobile Electronics
Chair Furniture
Table Furniture
Laptop Electronics

Expected Output:

Electronics 3
Furniture 2

Exercise 2

Find the **top occurring words** in a text file using MapReduce.

Exercise 3

Modify mapper to **ignore stop words**:

is
the
a
an

Real Industry Use Cases

MapReduce is used for:

- Log analysis

- Clickstream analysis
- Search indexing
- Recommendation systems
- Fraud detection

Companies using Hadoop ecosystem:

- Facebook
- Yahoo
- Amazon
- LinkedIn